Seed-VC

Zero-shot voice conversion by flow matching and in-context conditioning

A conceptual walkthrough of the `seed-uvit-whisper-small-wavenet` preset — grounded in the `burn-seedvc` pure-Rust reimplementation

---

Abstract. — Seed-VC re-renders a source recording in a target voice, like RVC and GPT-SoVITS before it, but it does so **without ever training on the target**: a single reference clip of a few seconds is the entire speaker specification. This document explains how that is possible. Two ideas carry it. The first is **flow matching** — the generative core learns a velocity field rather than a denoising schedule, which is what makes a handful of solver steps enough. The second is **in-context conditioning** — the reference is not compiled into weights but fed in beside the input, as a timbre vector and as a prefix the output is generated **after**, so the model continues the reference rather than imitating it. A third of this document is spent on what the released checkpoint contains and the released model never runs, because that gap is wide here and mistaking a fossil for a component is the failure mode that loads at one hundred per cent and sounds wrong.

Contents

1 Provenance: which Seed-VC this is

Everything below describes the **v1** preset `seed-uvit-whisper-small-wavenet`, mirrored from `Plachtaa/seed-vc` — within it `modules/diffusion_transformer.py`, `modules/flow_matching.py`, `modules/length_regulator.py`, `modules/wavenet.py`, `modules/campplus/` and `modules/bigvgan/`, with `inference.py` as the authority on how the pieces are wired together. The reference tree is read and never run; the Rust modules keep its field names and nesting so the published PyTorch `state_dict` maps onto the module tree parameter for parameter, which is the only way weight coverage can mean anything.

1.1 The licence, and what it did to this workspace

Upstream is GPL-3.0, and a port written from reading it is a derivative work. The licence therefore carries to everything that links `burn-seedvc`, and the `voice` workspace was relicensed from `MIT OR Apache-2.0` to `GPL-3.0-only` for exactly this reason — one line, since every crate inherits the workspace licence. `-only` rather than `-or-later` because upstream ships the bare GPL-3.0 text with no "or any later version" statement, and `-only` is the reading that is valid either way. Anything that must stay permissive cannot depend on this crate.

The networks Seed-VC itself borrows are permissively licensed and travel in the compatible direction: CAM++ comes from 3D-Speaker under Apache-2.0, BigVGAN from NVIDIA under MIT with Apache-2.0 alias-free resampling, and the residual quantiser mirrors descript-audio-codec under MIT. None of them widens the obligation beyond what Seed-VC already imposed.

1.2 Why this preset, and what v2 would need first

Seed-VC ships several presets, and this one was picked for a reason that outweighs the rest: **its content encoder is `openai/whisper-small`, which `burn-whisper` already loads.** The alternatives want either XLSR, a wav2vec2 variant nothing in this toolkit has, or — for the v2 CFM-plus-autoregressive pair — the ASTRAL-Quantization tokeniser, which is a port of its own before any of the rest can begin. Reaching v2 is therefore gated on a tokeniser, not on the diffusion model.

1.3 Three released files, not one

A reader hunting for a tensor in the wrong file loses an afternoon, so the split is worth stating before anything else: **two of this model's six networks are in nobody's checkpoint but their own.**

file	tensors	what is in it
`DiT_seed_v2_uvit_whisper_small_wavenet_bigvgan_pruned.pth`	302	the diffusion transformer and its WaveNet tail (255 under `net.cfm.module.estimator.`), the length regulator (22), and two fossils — the style encoder (18) and the quantiser (7). 110,035,232 parameters, 440 MB
`openai/whisper-small`	—	the content encoder. Only the encoder half is used; upstream does

		`del whisper_model.decoder`, and the port's loader lets the whole decoder land in `unused` on purpose
`campplus_cn_common.bin` (HF `funasr/campplus`)	937	the speaker encoder the released inference path actually conditions on. 815 applied, 0 missing; the remainder are one `num_batches_tracked` per norm, a training counter inference never reads
`nvidia/bigvgan_v2_22khz_80band_256x`	—	the vocoder, used exactly as NVIDIA released it and trained by nobody here

`burn-seedvc`'s `examples/load` prints coverage per prefix against all three, and that is deliberate rather than decorative: **a subtree nobody claims is indistinguishable from a subtree somebody forgot**, so every prefix has to appear with a number beside it, including the ones that turn out to be dead (§9).

2 What zero-shot conversion buys, and what it costs

RVC and GPT-SoVITS both answer "make this recording sound like Alice" by **training a model of Alice**: an hour of her speech, a few GPU-hours, and a `.safetensors` file that can convert into her voice and no other. Seed-VC answers it by **describing** Alice — a reference clip of between one and thirty seconds, analysed at run time, and nothing is written to disk at all.

That is a different bargain rather than a newer version of the same one. What is gained is obvious: a voice costs a recording instead of a training run, and the `seedvc` engine consequently has **no `train` subcommand anywhere in it**, which is the clearest structural difference between it and its two siblings. What is paid is fidelity to a specific voice — a fine-tune has seen an hour of Alice and a reference vector has seen four seconds — and a much larger inference-time model, since everything a fine-tune baked into weights now has to be computed per run.

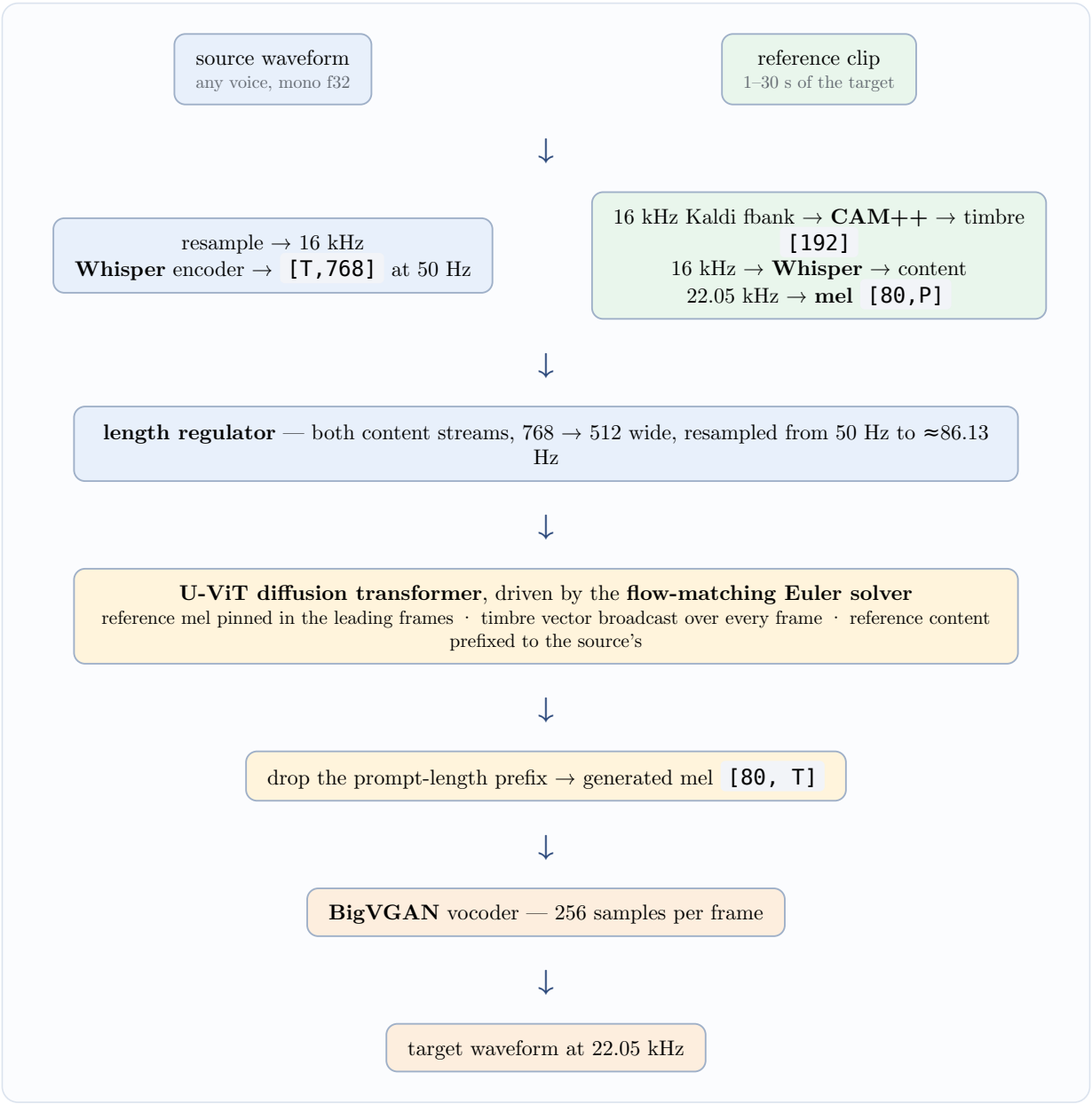

Figure 1: The end-to-end conversion path. The left column is the **source**, which supplies content only; the right column is the **reference**, which supplies everything about the voice. They meet inside the diffusion transformer and nowhere else.

The rest of this document unfolds that diagram: the solver first (§3), because it is what the transformer is **for**; then the analysis front ends (§4) and the rate change between them (§5); then the conditioning scheme that replaces training (§6); then the transformer (§7) and the vocoder (§8).

3 Flow matching, and why it replaces a diffusion schedule

The generative core is a **conditional flow-matching** model, upstream's `CFM`. It is worth being precise about what that means, because "diffusion transformer" is in the class's name and the sampling procedure is not a diffusion sampler.

3.1 The idea

A diffusion model prescribes a **forward corruption process** — a fixed schedule that mixes data with noise over a sequence of levels — and learns to reverse it one level at a time. The schedule is part

of the model: the sampler has to walk the ladder the trainer defined, and the number and spacing of its rungs are not free parameters.

Flow matching throws the ladder away. It picks a **path** between a simple distribution and the data distribution, and learns the **velocity field** whose flow transports one along that path to the other. Sampling then stops being a special procedure and becomes an ordinary initial-value problem: start at noise at $t = 0$, integrate

$$\frac{dx}{dt} = v(x, t) \quad \text{from} \quad t = 0 \quad \text{to} \quad t = 1,$$

and whatever the integrator lands on is a sample. The transformer **is** v.

Seed-VC picks the simplest path there is — a straight line. With z a draw from the Gaussian and x_1 a real mel, upstream's training pass forms

$$x_t = (1 - (1 - \sigma_{\min})t) \cdot z + t \cdot x_1, \quad u = x_1 - (1 - \sigma_{\min})z, \quad \sigma_{\min} = 10^{-6},$$

and regresses the network's output at a uniformly random t onto u. Two consequences follow, and both are the reason this architecture is fast where a diffusion model is not. **The regression target is constant along each path** — it does not depend on t at all — so there is no schedule to learn and none to reproduce at sampling time. And **the paths are straight**, so a first-order integrator has very little curvature to miss, which is what makes a handful of steps enough where a diffusion sampler with the same budget would still be visibly noisy.

> **None of that training pass is ported.** `burn-seedvc` implements the sampler and nothing else — `flow.rs` holds no weights at all, since the checkpoint's `net.cfm.module.estimator.*` tensors are the transformer's. The objective above is read off upstream and is exercised by nothing here, so treat it as background rather than as a description of running code. Two details of it are worth carrying anyway, because they explain the sampler's shape: the loss is an L_1 (`reg_loss_type: 'l1'`), and it is **masked to the generated region**, so the prompt frames contribute nothing to it — which is why the sampler pins those frames rather than integrating them.

3.2 The solver is explicit Euler, first order, and nothing better

Upstream's `solve_euler` is named for what it is, and the port keeps it: a uniform grid of `steps` intervals from 0 to 1, and at each one

$$x \leftarrow x + \Delta t \cdot v(x, t).$$

That is first-order accurate, so the error falls like 1/ steps and no faster — doubling the step count roughly halves it. It is an easy thing to misread, because the step count sits where a diffusion model's would and those often buy more per step. It does not here.

Because the solver is pure arithmetic over an arbitrary field, it can be tested **exactly**, against flows that can be written down by hand — a stronger check than the weight-coverage harnesses the rest of the crate leans on. Integrating $(dx)/(dt) = x$ from 1 towards the analytic e, the port's measured shortfall is:

steps	error	ratio to previous
4	0.2769	—
10	0.1245	—
50	0.0267	—
8, 16, 32	—	1.7–2.0 each time it doubles: first order

A second-order scheme would show roughly 4 in that last column. The trade is therefore linear in both directions and the choice of step count is a taste one: 4–10 steps is the real-time range, 25–50 buys audible polish, and upstream's own CLI defaults to 30, which is what the port's `Sampler::default` uses.

3.3 Classifier-free guidance, and the sign that catches everyone

The estimator is evaluated twice per step — once with the conditioning it was given, once with that conditioning zeroed — and the two velocities are combined:

$$v = (1 + w) \cdot v_{\text{cond}} - w \cdot v_{\text{uncond}}.$$

At $w = 0$ this is exactly the conditioned velocity, not the unconditional one. The scale measures how far **past** the conditioned prediction to extrapolate, away from the unconditional one, so zero means "no extrapolation" rather than "no conditioning". Upstream skips the second evaluation entirely in that case and so does the port, which makes the reduction exact rather than merely equal to within rounding. Upstream's default is $w = 0.7$.

Two implementation facts are load-bearing. The pair is evaluated as **one forward pass of twice the batch** rather than two passes — the branches then share every kernel launch, which is most of the cost at these sequence lengths — and that is only legal because nothing in the transformer reduces over the batch dimension. A normalisation taken across dimension 0 would silently blend the guided and unguided branches into each other and turn guidance into a smear, so the port pins the independence with a test that runs a row alone and against its pair.

The second is an asymmetry inherited deliberately. **Inference builds the unconditional branch by zeroing the raw conditioning**; training's counterpart, `class_dropout_prob: 0.1`, zeroes the same three signals but **after** the content projection has run. That projection carries a bias, so a zeroed content stream reaches the transformer as the bias rather than as zeros, and the two operations are not quite the same. The port reproduces upstream's version rather than the tidier one, because the released weights have only ever been driven the inference way and "fixing" it would put the port somewhere the checkpoint has never been evaluated.

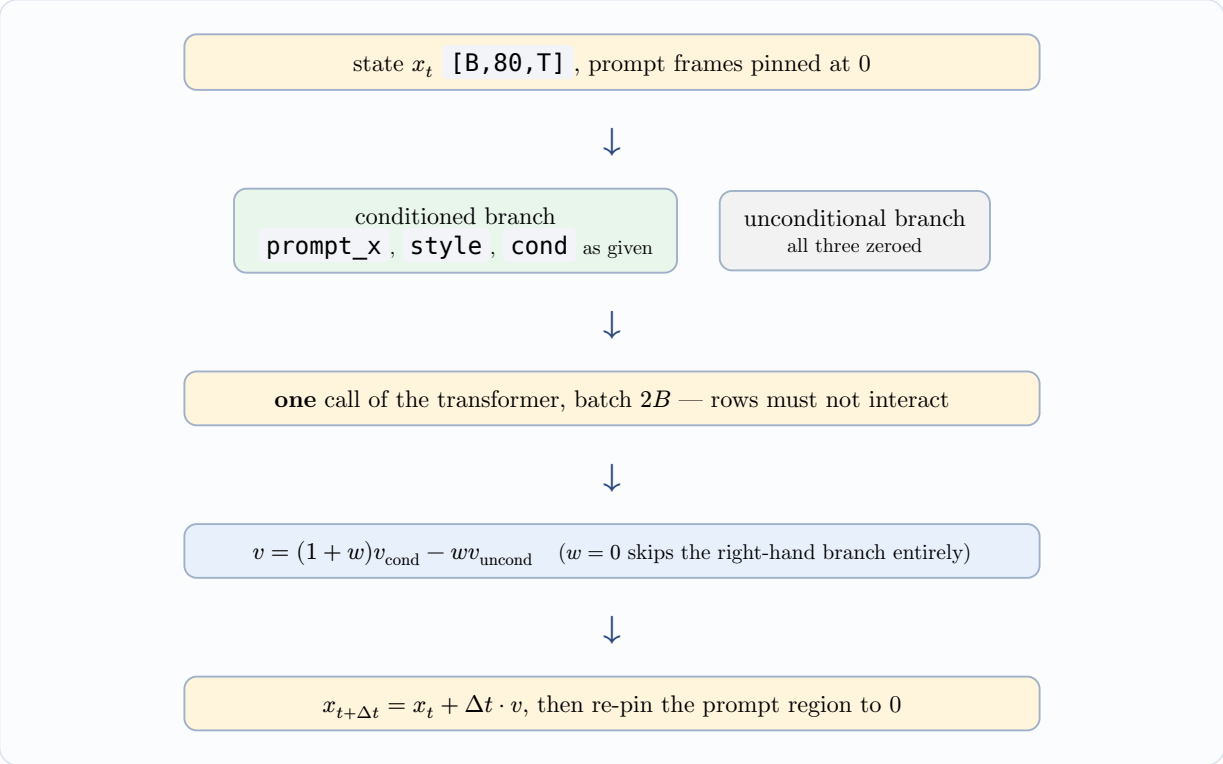

Figure 2: One Euler step. The prompt region of the state is held at zero throughout, so the integration only ever moves the frames after it; the reference mel travels alongside as `prompt_x` instead.

4 Three front ends at three rates

Everything upstream of the transformer is frozen analysis, and there are three separate front ends doing it. **They are not interchangeable, and every pairing of them produces 80 bands on a plausible frame grid** — which is exactly why substituting one for another runs happily, errors nowhere, and computes something else.

front end	rate	frames	transform
content (Whisper)	16 kHz	50 Hz	Whisper's own log-mel: centred STFT, **power** not magnitude, `log10` with a peak-relative floor and a $(x + 4)/4$ rescale
timbre (CAM++)	16 kHz	100 Hz	Kaldi filterbank, 80 bins, 25 ms window, 10 ms hop, `dither=0`, then mean-subtracted over time — and taking `[batch, frames, bins]`, the **opposite** axis order to everything else in the crate
mel (everything after)	22.05 kHz	≈86.13 Hz	the HiFi-GAN transform of `burn_vits::Spectral`: n_{fft} 1024, hop 256, 80 bands, magnitude, natural log

4.1 The content encoder is Whisper, unchanged

Seed-VC's content encoder **is** `openai/whisper-small`'s encoder — no port, no adapter layer, no fine-tune. Twelve layers over Whisper's log-mel produce 768-wide features at 50 Hz, and 768 is the load-bearing number: it is the length regulator's input width, and it is why this preset was the first target.

Whisper's front end is written out a third time in this workspace rather than reused, and the reason is worth recording because the two transforms look interchangeable. `burn_vits::Spectral` pads $(n_{\text{fft}} - \text{hop})/2$ where Whisper centres with $n_{\text{fft}}/2$; it takes magnitude where Whisper takes power; it

ends in a natural log where Whisper ends in `log10` with a floor eight decades below the window's own peak. **The frame counts agree**, so substituting one for the other shifts every feature by half a hop and rescales it — silently.

One structural consequence of using Whisper as-is: its encoder is fixed at a thirty-second window and a 1500-frame positional table, so a clip is zero-padded up to 30 s and the leading `samples / 320 + 1` frames are the real ones. **Audio longer than 30 s is truncated in the port.** Upstream chunks it with a five-second overlap and stitches the features back together; that path is not implemented, and it is the caller's job to notice.

4.2 The timbre encoder is CAM++, from somebody else's release

This is the network `inference.py` builds as `CAMPPlus(feat_dim=80, embedding_size=192)`, and **its output is the vector the transformer is actually conditioned on** — so without it there is no zero-shot conversion at all. Its weights are `campplus_cn_common.bin` from `funasr/campplus`, a 3D-Speaker model that has nothing to do with the Seed-VC checkpoint.

Its shape, since it is the least familiar network here:

FCM a 2-D convolutional ResNet front end that treats the filterbank as an image and downsamples **frequency** by 8 while leaving time alone, then folds the 32 channels and 10 remaining bins into one 320-channel sequence.

The TDNN stack a strided time-delay layer — the only thing that halves the frame rate, 100 Hz to 50 Hz — then three **CAM dense blocks** of 12, 24 and 16 layers. Dense means every layer's output is concatenated onto its input, so a block's width grows by `growth_rate` per layer and a transition layer halves it again afterwards.

Context-aware masking the "CAM", and the piece worth reading twice. Each layer computes a local convolution and multiplies it by a gate derived from two pooled summaries — the whole utterance's mean, plus a 100-frame segment average broadcast back over time. So every frame is scaled by what its neighbourhood and the utterance as a whole look like, which is how a speaker-level network suppresses content-level detail.

Statistics pooling mean and standard deviation over time, concatenated. This is what makes the reference **length** irrelevant — one second and thirty seconds condition the model with the same 192 numbers — and it is the premise zero-shot conversion rests on.

No numerical diff against upstream exists, and that should be said plainly. The layout is not guessed — upstream's source is available and the 937 tensors account for it exactly — but the port's check is only that the embedding is finite, repeatable, and moves when the spectral tilt of its input moves. That catches a network ignoring its input, which is the failure mode that looks healthiest; it would not catch a subtly mis-scaled one. The `eps = 1e-5` is PyTorch's `BatchNorm` default read off the constructor, and **no tensor in the checkpoint pins it**.

5 The length regulator: where 50 Hz becomes 86 Hz

One small convolutional stack sits between the frozen content encoder and the transformer, and it does exactly two things: it changes the **width** from Whisper's 768 to the transformer's 512, and it changes the **rate**.

The rate is the part three modules have to agree on. Whisper emits one frame per 320 samples at 16 kHz — 50 Hz. The mel the transformer predicts and the vocoder consumes runs at $22050/256 \approx 86.13$ Hz. This module is where one becomes the other, by nearest-neighbour interpolation onto a frame count **the caller supplies** — upstream's `ylens`, set to the mel's own frame count. The target length being an argument rather than a fixed ratio is also how upstream's `length_adjust` knob stretches or compresses the result.

> **`sampling_ratios` is not a ratio.** Upstream reads only its **length** — one `conv → GroupNorm → Mish` stage per entry — and never looks at the values. `[1, 1, 1, 1]` means four stages, not "rate unchanged", and a reader who takes the name at face value will conclude this module preserves the frame rate when **it is the only thing in the entire model that changes it**.

Its `GroupNorm` has a single group, which makes it a LayerNorm in everything but the parameter names — worth knowing only because Burn spells the pair `gamma` / `beta` and the checkpoint spells it `weight` / `bias`, so those keys are counted twice in a coverage report.

6 In-context conditioning: how a reference replaces a fine-tune

This is the section that explains the title. Everything in §3 would work just as well for a model that had been trained on the target voice; what makes Seed-VC zero-shot is **where the speaker information enters**.

6.1 The reference conditions the transformer three times

Reading `inference.py` from the top, the reference clip is analysed three ways and all three reach the transformer through different doors:

1. **As a timbre vector.** A Kaldi filterbank of the clip at 16 kHz through CAM++ gives 192 numbers (upstream's `style2`). They are broadcast over every frame of the window and concatenated into the transformer's input.
2. **As a mel prefix.** The clip's own 22.05 kHz mel is written into the leading frames of a full-width tensor (`prompt_x`), while the **matching frames of the state being integrated are pinned to zero for the whole solve**. So the model is shown real mel of the target voice sitting immediately before the region it is generating.
3. **As a content prefix.** The clip is **also** run through Whisper and the length regulator, and the result is concatenated in front of the source's content stream (`cat_condition = [prompt_condition, chunk_cond]`).

The third one is the one a reader skips, and it is what makes the other two mean something. Because both the reference's content **and** the reference's mel are present, and they are aligned, **the model is shown a complete worked example — this content, in this voice, looks like this — and then asked to continue it with the source's content.** It is not being asked to imitate a style vector. It is being asked to finish a sequence.

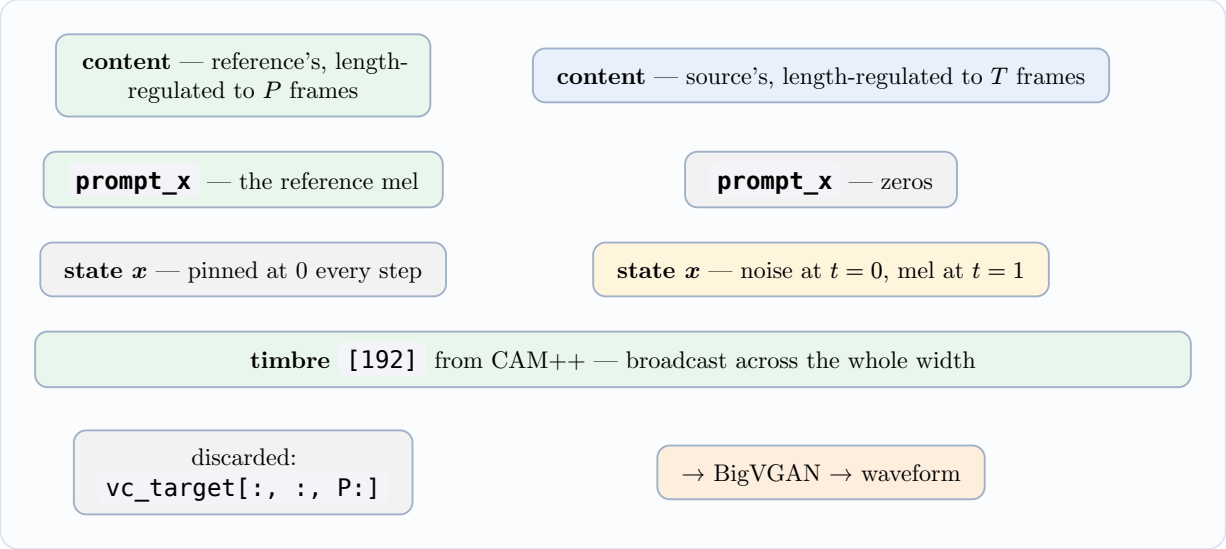

Figure 3: The generation window. The prompt region is written once and never integrated; the generated region is what the solver moves. The prefix is sliced off after decoding, which is why the transformer's window is longer than the output.

6.2 Why this is a fine-tune's replacement rather than its approximation

A fine-tuned model carries the target voice in its weights, and the cost of a new voice is a training run. Here the target voice is carried **entirely in the inputs**, so the cost of a new voice is one forward pass of CAM++, one of Whisper and one mel — seconds, not hours, and nothing written to disk. The generative weights are identical for every speaker who will ever be converted into, which is also why the released checkpoint is one file rather than a family.

Readers of `tts` will recognise the shape: GPT-SoVITS's `s1` stage generates by continuation in exactly this sense, and fails in exactly the way a continuation model fails when its prompt is incoherent. There is one happy difference. `s1` needs the reference's **transcript** supplied by the user, and a wrong one degrades the output quietly; here the reference's content stream is derived from the reference audio by the same frozen encoder that reads the source, so **the prompt cannot disagree with itself and there is no `--reference-text` trap to fall into**.

6.3 What the scheme costs at run time

The prefix is not free. It occupies frames of the same window the source has to fit into, and upstream's context budget is 30 s of mel — $22050/256 \times 30 \approx 2584$ frames — so `max_source_window = max_context_window - P`. **A longer reference therefore buys a better speaker specification with shorter source chunks**, and upstream truncates the reference at 25 s for that reason as much as any other. Long sources are generated chunk by chunk with a 16-frame overlap and a crossfade between consecutive waveform chunks; the port's own streaming layer is the engine's business rather than the network's.

The transformer's `block_size` of 8192 sits well above the 2584-frame budget, so it is not the binding constraint — but it **is** asserted in the port, because a sequence longer than the trained positional range is the kind of thing that produces degraded output rather than an error.

7 The U-ViT diffusion transformer

255 of the checkpoint's 302 tensors are here. This is the only generative network in Seed-VC; everything either side of it prepares its conditioning or renders its output.

7.1 Everything enters through one projection

The transformer's input at each frame is a concatenation of four things — the noisy mel (80), the prompt mel (80), the projected content (512) and the timbre vector (192) — and $80 + 80 + 512 + 192 = 864$, which is exactly what `cond_x_merge_linear [512, 864]` records. **The whole of the model's conditioning enters through that one projection**, and that is precisely why classifier-free guidance can be implemented by zeroing inputs: there is no other door.

The flow time t enters somewhere else entirely, through **adaptive layer normalisation**, and is the only conditioning that does.

7.2 Three shapes carry the architecture

Each of these reads naturally as its more common cousin, and each such reading gives a port that loads at 100% and produces plausible-looking rubbish — a failure this repository has shipped once already, in Whisper's causal mask.

`project_layer [1024, 512]` on every norm adaptive layer normalisation. The conditioning vector — here the flow-time embedding, and nothing else — is projected to 2×512 and split into a scale and a shift applied around the norm. A plain norm would drop the model's only sense of where along the trajectory it is. What sits inside is **RMSNorm**, not LayerNorm: one `weight`, no bias, no mean subtraction. The output head splits its two chunks in the **opposite order** to the blocks, because upstream writes them that way in the two places; swapping them is a silent quality loss, not a crash.

`w1`, `w2`, `w3` with `w1` and `w3` both `[1536, 512]` a gated feed-forward, $w_2(\text{silu}(w_1 x) \odot w_3 x)$ — SwiGLU. A two-matrix MLP would consume the same tensors in the same order and simply compute something else. The 1536 is not arbitrary: it is `find_multiple(2/3 · 4 · 512, 256)`.

`skip_in_linear [512, 1024]` per layer and `skip_linear [512, 592]` at the top two different skip schemes, not one. The per-layer one is the **U-ViT** connection. The top-level one is the **long** skip, concatenating the input mel onto the transformer's output — hence $592 = 512 + 80$.

7.3 The U, and why the depth is odd

Layers 0–5 emit their output onto a stack; layers 7–12 pop one back off and concatenate it, so layer 7 receives layer 5′s and layer 12 receives layer 0′s. Layer 6 is the bottom of the U and does neither. **Upstream's depth of 13 is odd, which is exactly what makes the emitting and receiving lists the same length** — an even depth would leave one emitted tensor unclaimed, and upstream would not error either. The port's tests use a depth of 5 for the same reason.

Every block carries a `skip_in_linear`, including the seven that never receive a skip, because upstream builds it from a config flag rather than from the block's index. Those tensors are in the checkpoint and are loaded; they are simply never called.

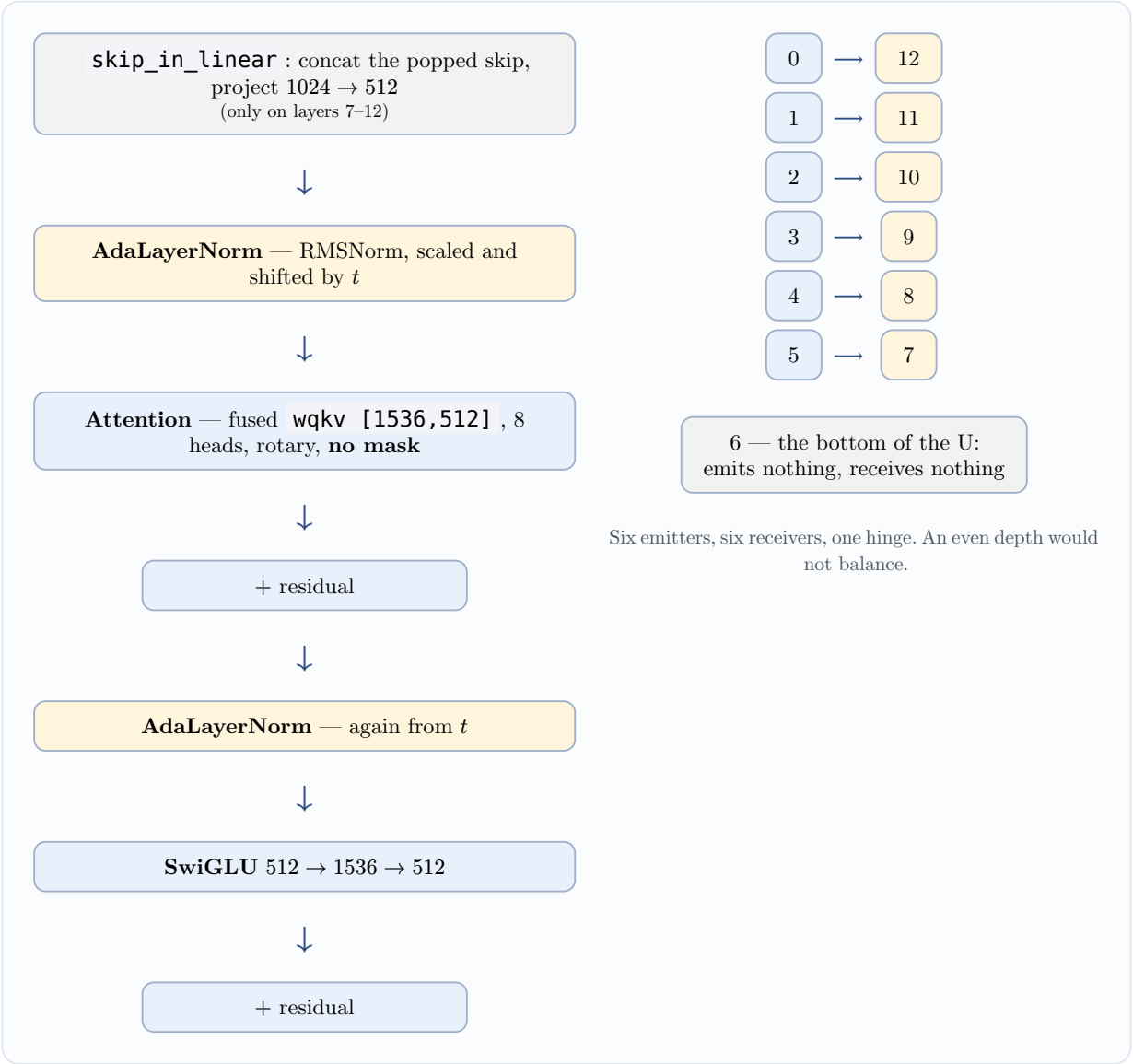

Figure 4: Left: one pre-norm block, with the flow-time embedding entering through both adaptive norms and nowhere else. Right: the U-ViT skip pattern over the released depth of 13.

7.4 There is no causal mask, and adding one is the classic mistake

`is_causal: false`. This is a denoiser, not a language model: it sees the whole utterance at once and its attention is unmasked. Burn's `triu_mask` / `tril_mask` are named for the triangle they **keep**, and a wrong mask feeds a full row of $-\infty$ into softmax, which is `NaN` rather than an error and reaches every output — and Burn's approximate comparisons treat `NaN` as equal to `NaN`, so finiteness has to be asserted separately. The port's tests do.

Upstream **does** build one mask, from `x_lens`, but it is a **padding** mask over keys and is all ones for a single clip. Inference only ever passes one clip — the guidance pair being the same clip twice — so **the port does not model it**, and the argument is kept on the trait so that a padded batched trainer would have somewhere to put it back. That is an untested path, named as one.

7.5 Rotary positions, in the convention that is not the common one

The pairs rotated by the rotary embedding are **adjacent, not half-and-half**. Upstream's transformer is Meta's `gpt-fast`, which reshapes the last dimension to (head_dim $/2, 2$) and rotates (x_0, x_1), (x_2, x_3) and so on, where the split-halves convention of Llama-style code rotates

$\left(x_i, x_{i+\text{ head\_dim }/2}\right)$. Both are called "RoPE", they are not interchangeable, and choosing wrongly costs nothing at load time and everything at inference. The port pins the convention with a test rather than inferring it from a shape, because no shape reveals it.

The attention here is written out rather than reused from `burn-vits`, and the reason generalises: VITS's block always carries relative-position embeddings this checkpoint has no tensors for, keeps `conv_q` / `conv_k` / `conv_v` as separate projections where this is one fused `wqkv`, and has no rotary embedding at all. Bending it would change a module that RVC and GPT-SoVITS both depend on to suit a third caller agreeing with neither.

7.6 The WaveNet tail

The transformer's output does not go straight to the mel. It passes through a `conv1` projection, eight gated weight-normalised convolutions, and a residual back onto the projection — so **the WaveNet refines the transformer's output rather than replacing it** — before the output head and a final width-1 convolution down to 80 bands.

Three things distinguish it from `burn_vits::Wn`, which is otherwise the same `fused_add_tanh_sigmoid_multiply` shape:

- **It is conditioned per layer.** VITS adds a single broadcast vector to every layer; this one slices a per-layer window out of `cond_layer`'s $2 \times 512 \times 8 = 8192$ output channels.
- **It is conditioned by a second timestep embedder.** `t_embedder2` has the same architecture as `t_embedder` and separate weights: the transformer half and the WaveNet half are conditioned independently, and reusing one embedding for both would load fine and quietly halve the conditioning capacity.
- **Its convolutions reflect-pad.** Upstream's `SConv1d` is handed a `padding` argument by its call site and **never forwards it** — it disappears into `**kwargs`, the wrapped convolution is built unpadded, and `SConv1d.forward` reflect-pads the input instead. Reading the call site rather than the wrapper is how a port acquires zero padding, which is not an error anywhere: it simply pulls the first and last frames of every layer towards silence, eight times over.

`dilation_rate` is 1 on this preset, so the receptive field grows **linearly** rather than exponentially — worth knowing before assuming a WaveNet-shaped module has WaveNet's usual reach. Upstream's `x_mask` multiplications and its `p_dropout: 0.2` are both absent from the port for the reasons above: the first is all ones for a single clip, the second is a training-time term.

8 BigVGAN: the vocoder nobody here trained

80 mel bands in at 22.05 kHz, waveform out. `nvidia/bigvgan_v2_22khz_80band_256x` is used exactly as released, and its upsample rates multiply to 256 — matching the mel hop, so one mel frame becomes 256 samples and the frame grid the whole model agrees on carries through to the waveform.

Structurally it is HiFi-GAN: a width-7 pre-convolution, six transposed-convolution upsample stages, three residual blocks averaged after each, a width-7 post-convolution to one channel. Two things make it BigVGAN.

8.1 The snake activation, and which variant this is

Every leaky-ReLU of HiFi-GAN is replaced by a periodic activation with **learned per-channel parameters**. This checkpoint's `config.json` says `"activation": "snakebeta"` with `"snake_logscale": true`, so the function is

$$\text{snakebeta}(x) = x + \frac{1}{e^{\beta} + 10^{-9}} \cdot \sin^2(e^{\alpha} \cdot x)$$

with α and β **separate** per-channel tensors. The checkpoint agrees independently: every activation site carries an `alpha` and a `beta` of equal width, which the α-only `Snake` variant would not. Two ways of getting this wrong load at 100% and produce a plausible waveform — using α where β belongs, which is right only where the two coincide and after training they do not; and dropping the exponential, which on a log-scale checkpoint whose values sit near zero turns a frequency of about 1 into about 0 and multiplies the periodic term by roughly 10^9.

8.2 Anti-aliasing, and filters that are both computed and stored

$\sin^2$ doubles the bandwidth of whatever it is fed, so applying it at the signal rate folds everything above Nyquist back down as audible aliasing. BigVGAN wraps each activation in 2× upsample → activate → 2× downsample, both resampling steps being grouped convolutions with a Kaiser-windowed sinc low-pass (cutoff 0.25, transition half-width 0.3, 12 taps). That wrapper is the "anti-aliased multi-periodicity" the paper is named for.

Those kernels are computed rather than learned — and the checkpoint stores them anyway. Upstream registers each with `register_buffer`, which is persistent by default, so all 218 are in the file: 18 residual blocks × 6 activations × 2 resamplers, plus the post-activation's pair. Being a deterministic function of (cutoff, half-width, taps), all 218 are the same twelve numbers. The port **derives** them, so the module is correct with no checkpoint at all, and then lets the file overwrite them — which keeps the coverage report at zero unused and turns the stored copies into a free check on the derivation.

Two smaller v2 flags decide things no shape could reveal. `use_tanh_at_final: false` means the output is a hard clamp to ±1 rather than a `tanh`, and these weights were trained against the clamp — `tanh` would compress every peak instead of passing it. `use_bias_at_final: false` means the final convolution's bias tensor is **absent from the checkpoint** rather than zero, which is why that one convolution is local to the crate instead of `burn-vits`'s.

Because a vocoder reconstructs phase from scratch, **a sample-wise difference between two runs proves nothing either way** — the port's check is that the output's energy envelope correlates with the input's, reported beside a shuffled control and a spectral-flatness figure, on the same reasoning `burn-gptsovits`'s `s2` reconstruction check uses.

9 The fossils: what the checkpoint holds and the model never runs

This section exists because the gap is unusually wide here, and because **a reader who assumes the checkpoint's contents are the model will wire inference to the wrong network and hear something plausible**. Upstream's `commons.build_model` assembles exactly two things — `cfm` and `length_regulator` — and `load_checkpoint` iterates over what `build_model` returned. Everything else in the file is read straight past.

subtree	tensors	status
`net.style_encoder.*`	18	**a fossil of the training-time model.** `build_model` never constructs it; inference builds a separate CAM++ instead. §4 is its live replacement

`net.vq.*`	7	a residual quantiser over the content stream. `vector_quantize: false`, and nothing in upstream's current tree builds it
`length_regulator.embedding` + `mask_token`	2	the **discrete** content path. `is_discrete: false`, so content stays continuous and the 2048-entry codebook is never indexed
`x_embedder`	1	allocated by `DiT.__init__`, never called by `DiT.forward` — the mel enters through `cond_x_merge_linear` instead
`cond_embedder`	1	the discrete-content codebook (1024 × 512). `content_type: 'discrete'` in the config is a **claim the code overrides**: `cond_in_module` is pinned to the continuous projection with the discrete branch commented out
`content_mask_embedder`	1	constructed, never referenced — guidance zeroes the conditioning rather than substituting a learned token
`f0_embedder`	1	`f0_condition: false`, and the released source has no `f0_embedder` at all. A fossil of a variant that did
`input_pos`	1	a buffer holding `arange(8192)` — the positions themselves. Rotary tables are recomputed from the sequence length, so its values are never read

All of them are ported anyway, and that is a deliberate policy rather than completionism: dropping a subtree would report false `unused` entries and hide a real one in the noise, and the whole value of a coverage report is that every prefix has a number beside it. The rule the port follows is "port faithfully, document what is live".

9.1 The style encoder is also the one thing here that is genuinely guessed

`net.style_encoder` is two pointwise convolutions, two gated convolutions, one round of self-attention and an average over time, consuming an 80-band mel and producing 192 numbers. It looks exactly like the network that ought to be supplying the timbre vector, and it is not: the preset's own config gives it away (`style_encoder.campplus_path`), and `inference.py` passes CAM++'s output as `style2`.

Worse for anyone trying to verify it, **the class was never released**. It is in no commit of the public repository, and `build_model` has never referenced it. The port's layout is read off tensor shapes and off the lineage the names give away — StyleSpeech's `MelStyleEncoder`, rewritten with convolutions throughout, with VITS's attention block pasted in (`conv_q` / `conv_k` / `conv_v` / `conv_o` are that class's field names verbatim). Three things the checkpoint cannot settle, all of which load perfectly either way:

- **the head count** — no shape depends on it, since there are no relative-position embeddings to size. The port uses 2, inherited from StyleSpeech's `style_head=2`, and **it is a guess**.
- **the attention scale** — VITS divides by $\sqrt{k_{\text{channels}}}$, StyleSpeech by $\sqrt{d_{\text{model}}}$. The port follows VITS because the projection names say the VITS block was the one pasted in. With 2 heads over 512 channels the two differ by a factor of 4.
- **the residual around the attention** — StyleSpeech has one inside its block, VITS's has none and adds it in the caller. Kept, since the caller here **is** StyleSpeech's.

None of this is verified numerically and it cannot be from this checkpoint alone: there is no reference implementation to diff against and no downstream consumer whose output would visibly

degrade. Treat that forward pass as untested and its coverage number as covering the layout only. Wiring inference to it would feed the transformer a timbre vector Seed-VC was never conditioned on.

9.2 The quantiser, and where its dimensions came from

`net.vq` is a descript-audio-codec residual vector quantiser: project the 768-wide content down to **8 dimensions**, snap each frame to its nearest entry of a 1024-entry codebook, project back up. The low-dimensional bottleneck is the whole trick — a codebook only helps if its entries are dense in the space they cover, and 1024 points are hopeless in 768 dimensions and reasonable in 8. **The projections are the easy thing to get backwards**: `in_proj` is $768 \to 8$, `out_proj` is $8 \to 768$, and the codebook lives in the narrow space between them.

Its dimensions are read off the checkpoint's shapes rather than from a config, because `config.yml` has no `vq` section at all — which is the same evidence that says it is not built. There is one quantiser in the stack, so "residual" describes the architecture rather than anything that happens.

10 A note on dimensions

quantity	value	meaning
content dim	768	`whisper-small`'s `d_model`; the length regulator's input
`hidden_dim`	512	transformer width, and the length regulator's output
`style_dim`	192	the CAM++ embedding — the whole speaker specification
merge input	864	$80 + 80 + 512 + 192$: every conditioning signal, one projection
long skip	592	$512 + 80$: the transformer's output plus the input mel
depth / heads	13 / 8	odd depth by necessity (§7.3); head dim 64
SwiGLU hidden	1536	`find_multiple(2/3 · 4 · 512, 256)`
`block_size`	8192	longest sequence the positional scheme admits
WaveNet	8 layers, k=5, d=1	`cond_layer` is $2 \times 512 \times 8 = 8192$ channels
mel	80 bands	n_{fft} 1024, hop 256, at 22 050 Hz
content frame rate	50 Hz	one frame per 320 samples at 16 kHz
mel frame rate	≈86.13 Hz	22050/256 — what the regulator resamples **to**
fbank frame rate	100 Hz	10 ms hop, halved to 50 Hz by the first TDNN
vocoder upsample	$4 \cdot 4 \cdot 2 \cdot 2 \cdot 2 \cdot 2$	product = 256 = the mel hop
vocoder width	1536	initial channels, halved at each of the six stages
solver	30 steps, $w = 0.7$	upstream's `inference.py` defaults
context budget	≈2584 frames	30 s of mel, **shared** between prompt and source

11 Inference recap, and what is deliberately not modelled

The reverse path in one paragraph: analyse the reference three ways (CAM++ timbre, Whisper content, 22.05 kHz mel), analyse the source once (Whisper content), length-regulate both content streams onto the mel grid, concatenate the reference's in front of the source's, place the reference mel in the leading frames and pin the state's matching frames to zero, integrate the velocity field for thirty Euler steps with guidance 0.7, slice the prompt-length prefix off the result, and vocode. **Content came from the source; everything about the voice came from a recording the model has never been trained on.**

Four things a reader should not assume are present, each named as an untested or absent path rather than left to be discovered:

- **The key-padding mask is not modelled.** It is all ones for the one batch shape inference ever builds, and the guidance pair is the same clip twice. A padded batched trainer would have to reinstate it, in the transformer and in the WaveNet both.
- **Whisper's 30 s chunking is not implemented.** The port truncates; upstream chunks with a five-second overlap and stitches the features.
- **The F_0-conditioned variant is absent.** `f0_condition: false` on this preset, so the pitch extractor, the 44.1 kHz rate and the `f0_embedder` are all outside the port. That is not a gap — upstream never constructs them here either.
- **There is no training loop, and no plan for one.** Seed-VC's premise is that there is nothing to train, so unlike `rvc` and `tts` this engine has no `train` subcommand and `docs/training.md` has nothing to say about it. The flow-matching objective of §3.1 is documented as background only.

The one place a numerical check on the whole is still owed is the same place it was owed for every other port here: **weight coverage says the module tree matches the checkpoint and says nothing about whether the forward pass computes the right thing.** BigVGAN has an envelope check, the solver has exact tests against hand-integrable fields, and CAM++ has a sensitivity check that would catch a network ignoring its input. An end-to-end check — convert a clip, then compare what `stt` transcribes from the result against what it transcribes from the source — is the counterpart of the round-trip `tts` uses, and is the honest way to close the gap.

Concepts anchored to the `voice` implementation: `burn-seedvc` (the network), `seedvc-core` (the engine around it), `burn-whisper` (the content encoder), `burn-vits` (the mel and the weight-normalised convolutions). Preset `seed-uvit-whisper-small-wavenet`, weight-compatible with `DiT_seed_v2_uvit_whisper_small_wavenet_bigvgan_pruned.pth`, `campplus_cn_common.bin` and `nvidia/bigvgan_v2_22khz_80band_256x`. Ported from Seed-VC (GPL-3.0), read as a reference and never run.